Experiment 6 - Jenkins and Github

1. Prepare Your Project on GitHub

Ensure your Java project (either Maven or Gradle) is hosted on GitHub. For this experiment, we will use the following public example repositories:

- **Maven Project:** https://github.com/manjushree-devops/jenkins-maven-demo.git
- **Gradle Project:** https://github.com/manjushree-devops/jenkins-gradel-demo.git

  Note: You are welcome to use your own GitHub repository if you prefer. If you use the links provided above, you will not need to configure credentials as they are public repositories, and the default branch to build is `main`.

Make sure your project has a build file (`pom.xml` for Maven or `build.gradle`/`build.gradle.kts` for Gradle) that can compile your code and generate JUnit test reports (typically XML files).

Experiment: Integration of GitHub with Jenkins

Aim

To create a simple Java Maven project locally, push it to GitHub using Git commands, and integrate the repository with Jenkins to build and execute the project.

Tools Required

- Ubuntu / Linux Terminal
- Git
- GitHub Account
- Java JDK
- Maven
- Jenkins

Procedure to Create Project and Push to GitHub

1. Create a Project Directory

mkdir jenkins-devops-demo

cd jenkins-devops-demo

2. Create the Maven Project Structure

mkdir -p src/main/java/com/example

mkdir -p src/test/java/com/example

```
MINGW64:/c/Users/HP/jenkins-devops-demo
HP@MANJUSHREENAVEEN MINGW64 ~
$ mkdir jenkins-devops-demo
cd jenkins-devops-demo

HP@MANJUSHREENAVEEN MINGW64 ~/jenkins
-devops-demo
$ mkdir -p src/main/java/com/example

HP@MANJUSHREENAVEEN MINGW64 ~/jenkins
-devops-demo
$ nano pom.xml

HP@MANJUSHREENAVEEN MINGW64 ~/jenkins-devops-demo
$ nano src/main/java/com/example/App.java

HP@MANJUSHREENAVEEN MINGW64 ~/jenkins
-devops-demo
$ git init
Initialized empty Git repository in C
:/Users/HP/jenkins-devops-demo/.git/
```

3. Create pom.xml

nano pom.xml

4. Create Java Program

nano src/main/java/com/example/App.java

5. Create JUnit Test File

nano src/test/java/com/example/AppTest.java

6. Initialize Git Repository

git init

7. Add Files to Git

git add .

8. Commit Files

git commit -m "Initial DevOps Maven project"

9. Create GitHub Repository named: jenkins-maven-demo

10. Connect Local Repository to GitHub

git remote add maven https://github.com/yourusername/jenkins-maven-demo.git

11. Push Files to GitHub

git push maven master

```
MINGW64:/c/Users/HP/jenkins-devops-demo

HP@MANJUSHREENAVEEN MINGW64 ~/jenkins
-devops-demo (master)
$ git remote add maven https://github
.com/manjushree-devops/jenkins-maven-
demo.git

HP@MANJUSHREENAVEEN MINGW64 ~/jenkins
-devops-demo (master)
$ git push maven master
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threa
ds
Compressing objects: 100% (4/4), done
.
Writing objects: 100% (9/9), 827 byte
s | 275.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0)
, pack-reused 0 (from 0)
To https://github.com/manjushree-devo
```

```
MINGW64:/c/Users/HP/jenkins-devops-demo
Initialized empty Git repository in C
:/Users/HP/jenkins-devops-demo/.git/

HP@MANJUSHREENAVEEN MINGW64 ~/jenkins
-devops-demo (master)
$ git add .
warning: in the working copy of 'pom.
xml', LF will be replaced by CRLF the
 next time Git touches it
warning: in the working copy of 'src/
main/java/com/example/App.java', LF w
ill be replaced by CRLF the next time
 Git touches it

HP@MANJUSHREENAVEEN MINGW64 ~/jenkins
-devops-demo (master)
$ git commit -m "Initial DevOps Maven
 project"
[master (root-commit) 46326fe] Initia
l DevOps Maven project
 2 files changed, 23 insertions(+)
```

Program Files

1. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 http://maven.apache.org/xsd/maven-4.0.0.xsd">

 <modelVersion>4.0.0</modelVersion>

 <groupId>com.example</groupId>
 <artifactId>devops-demo</artifactId>
 <version>1.0-SNAPSHOT</version>
```

```
 <dependencies>
  <dependency>
   <groupId>junit</groupId>
   <artifactId>junit</artifactId>
   <version>4.13.2</version>
   <scope>test</scope>
  </dependency>
 </dependencies>

</project>
```

2. App.java

```
package com.example;

public class App {

    public static void main(String[] args) {

        System.out.println("Welcome to DevOps Lab --- Manjushree.G");

    }

}
```

3. AppTest.java (JUnit Test File)

```
package com.example;

import org.junit.Test;
import static org.junit.Assert.*;

public class AppTest {

    @Test
    public void testMessage() {
        String expected = "Welcome to DevOps Lab --- Manjushree.G";
        assertEquals(expected, "Welcome to DevOps Lab --- Manjushree.G");
    }
}
```

Expected Output

Welcome to DevOps Lab --- Manjushree.G

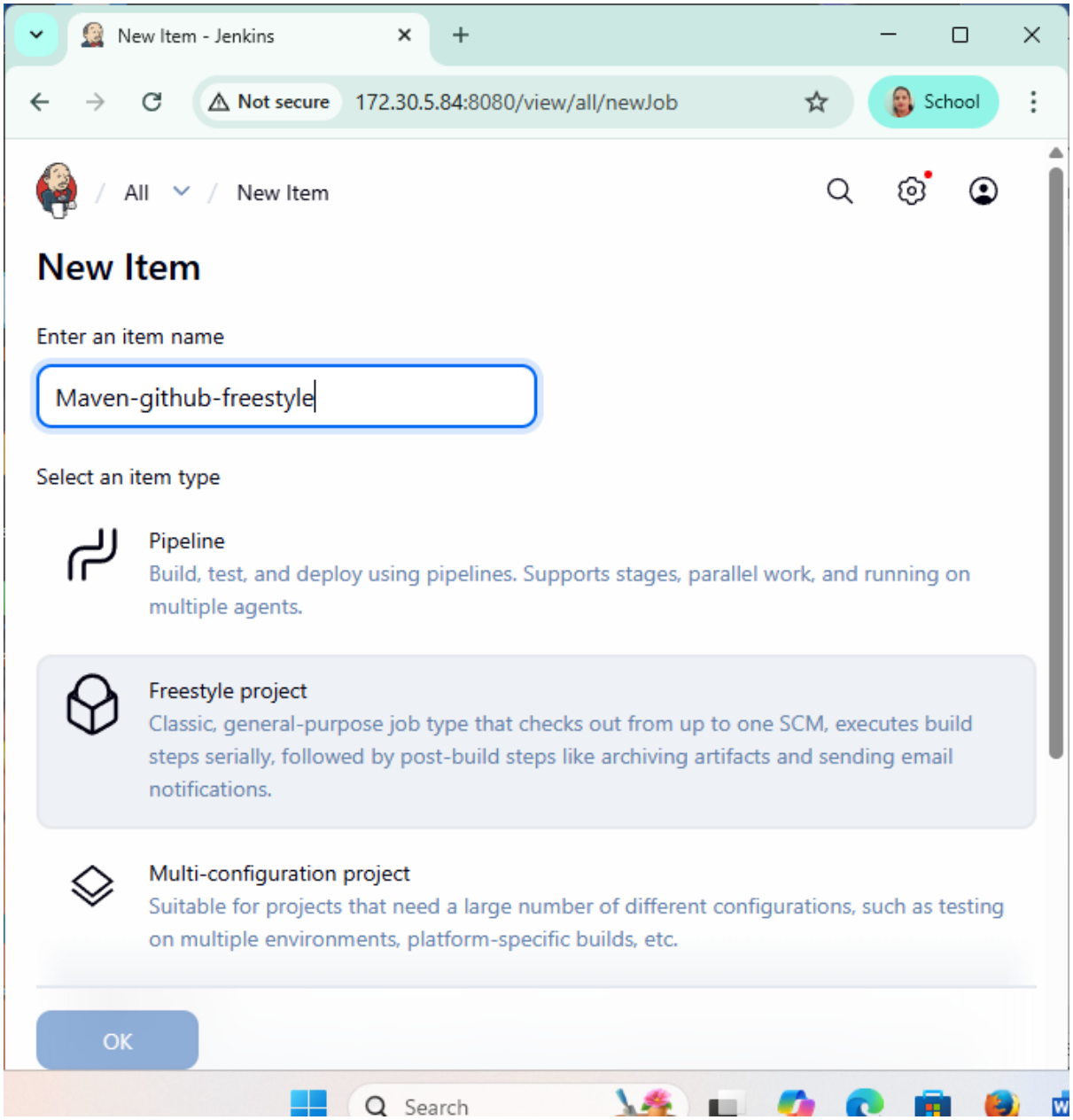
2. Log in to Jenkins

Open your web browser and go to your Jenkins instance (e.g., `localhost:8080`). Log in with the administrator credentials you created during the initial setup.

---

3. Create a New Jenkins Job (Freestyle Project)

1. On the Jenkins dashboard, click "New Item" (or "Create a job" if it's a fresh installation).
2. Enter an item name (e.g., `Maven-GitHub-Freestyle` or `Gradle-GitHub-Freestyle`).
3. Select "Freestyle project".
4. Click "OK".



---

4. Configure Source Code Management (SCM) for Freestyle

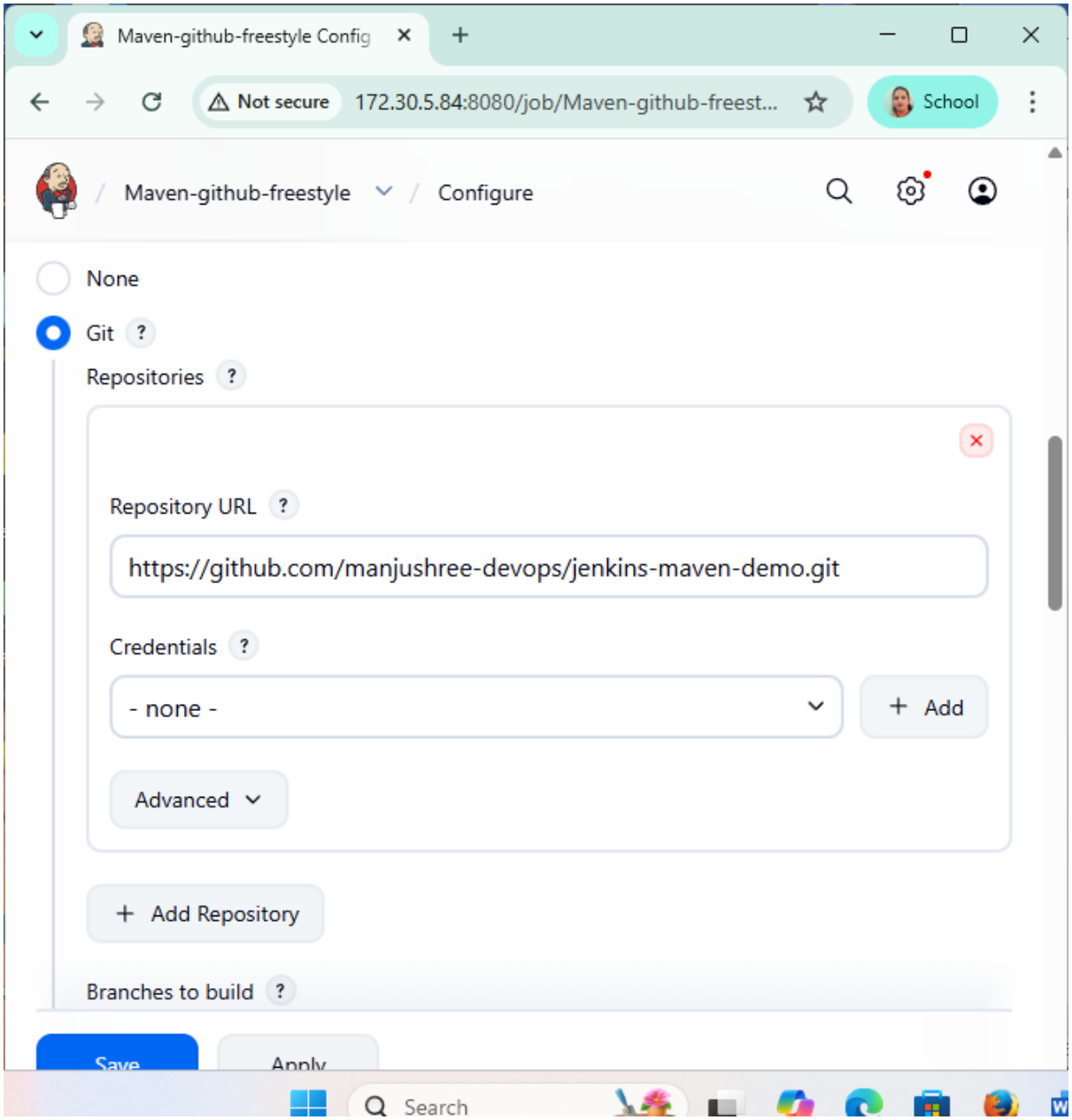
In the job configuration page for your Freestyle project:

1. Scroll down to the "Source Code Management" section.
2. Select "Git".
3. In the "Repository URL" field, enter the URL for your chosen GitHub repository (either one of the examples provided or your own).

   Maven Project: https://github.com/manjushree-devops/jenkins-maven-demo.git

Gradle Project: https://github.com/manjushree-devops/jenkins-gradel-demo.git

4. If you are using your **own private repository**, you will need to add credentials. Click "Add" next to "Credentials" and provide your GitHub username and password or a personal access token. If using the provided public repositories, no credentials are needed.
5. Under "Branches to build", ensure `*/main` is specified. If your repository uses a different default branch (like `*/master`), adjust this accordingly. For the provided public repositories, `*/main` is correct.



---

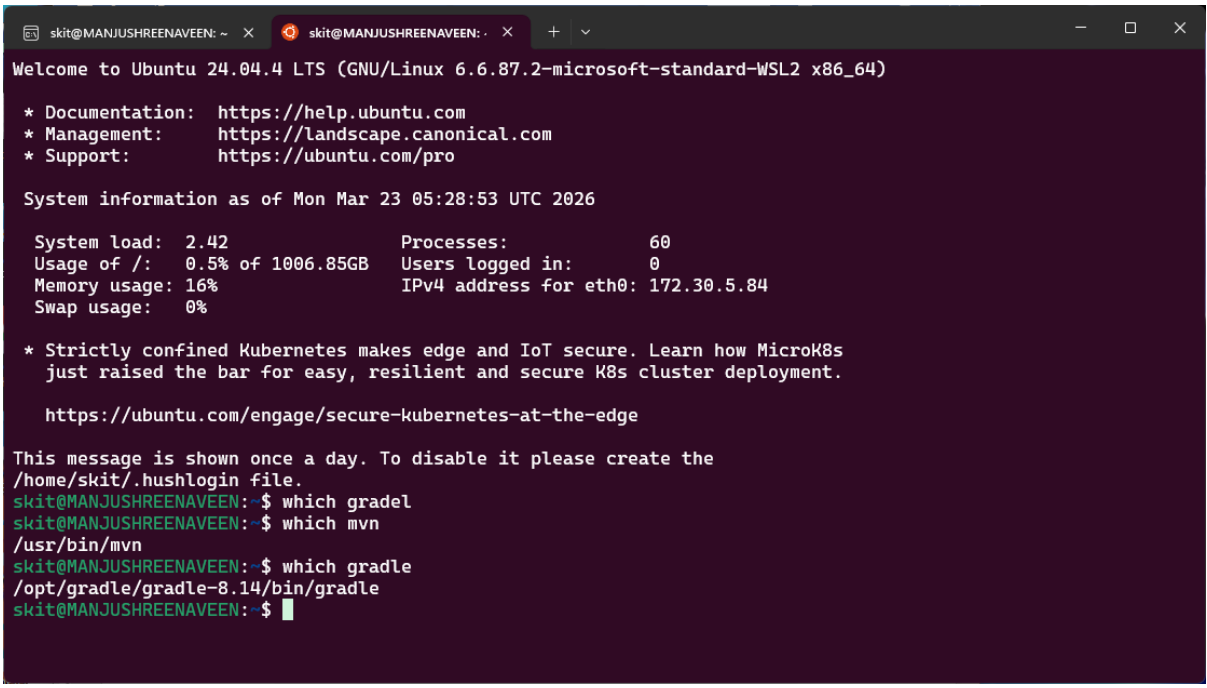
5. Configure the Build Step for Freestyle

This is where you tell Jenkins how to build your project using Maven or Gradle.

- **If you are building a Maven Project:**
    1. Scroll down to the "Build" section.
    2. Click "Add build step".
    3. Select "Execute shell" (or "Execute Windows batch command" on Windows).
    4. In the "Command" area, enter the full path to your Maven executable followed by the goals you want to run (e.g., `clean install`).

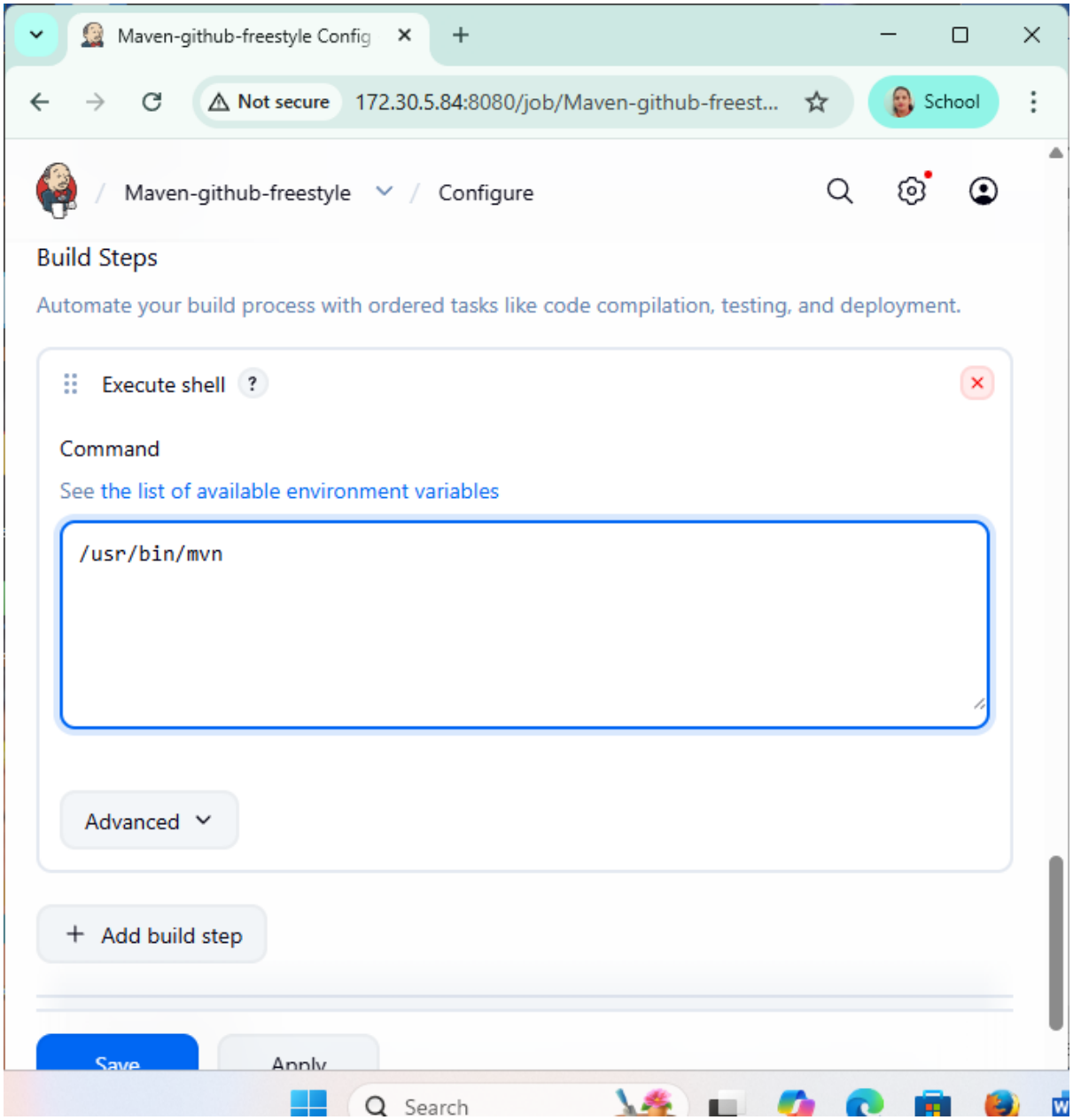
       To find the full path to your `mvn` executable on open terminal in Ubuntu and run:

       ```
       which mvn
       ```



       The output will give you the full path (e.g., `/usr/bin/mvn` or `/opt/maven/bin/mvn`). Use this path in the command below.

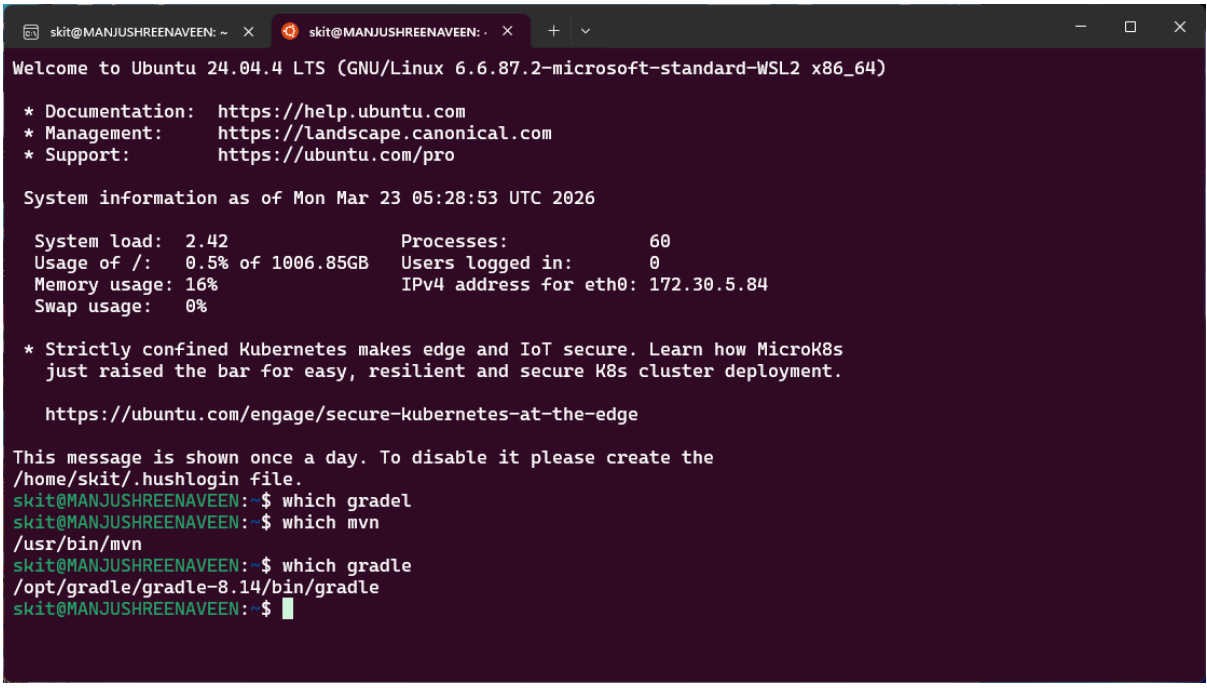
       ```
       #/path/to/your/maven/bin/mvn clean install
       # Example:
       /usr/bin/mvn clean install
       ```



- **If you are building a Gradle Project:**
  1. Scroll down to the "Build" section.
  2. Click "Add build step".
  3. Select "Execute shell".
  4. In the "Command" area, enter the full path to your Gradle executable followed by the tasks you want to run (e.g., `build`).

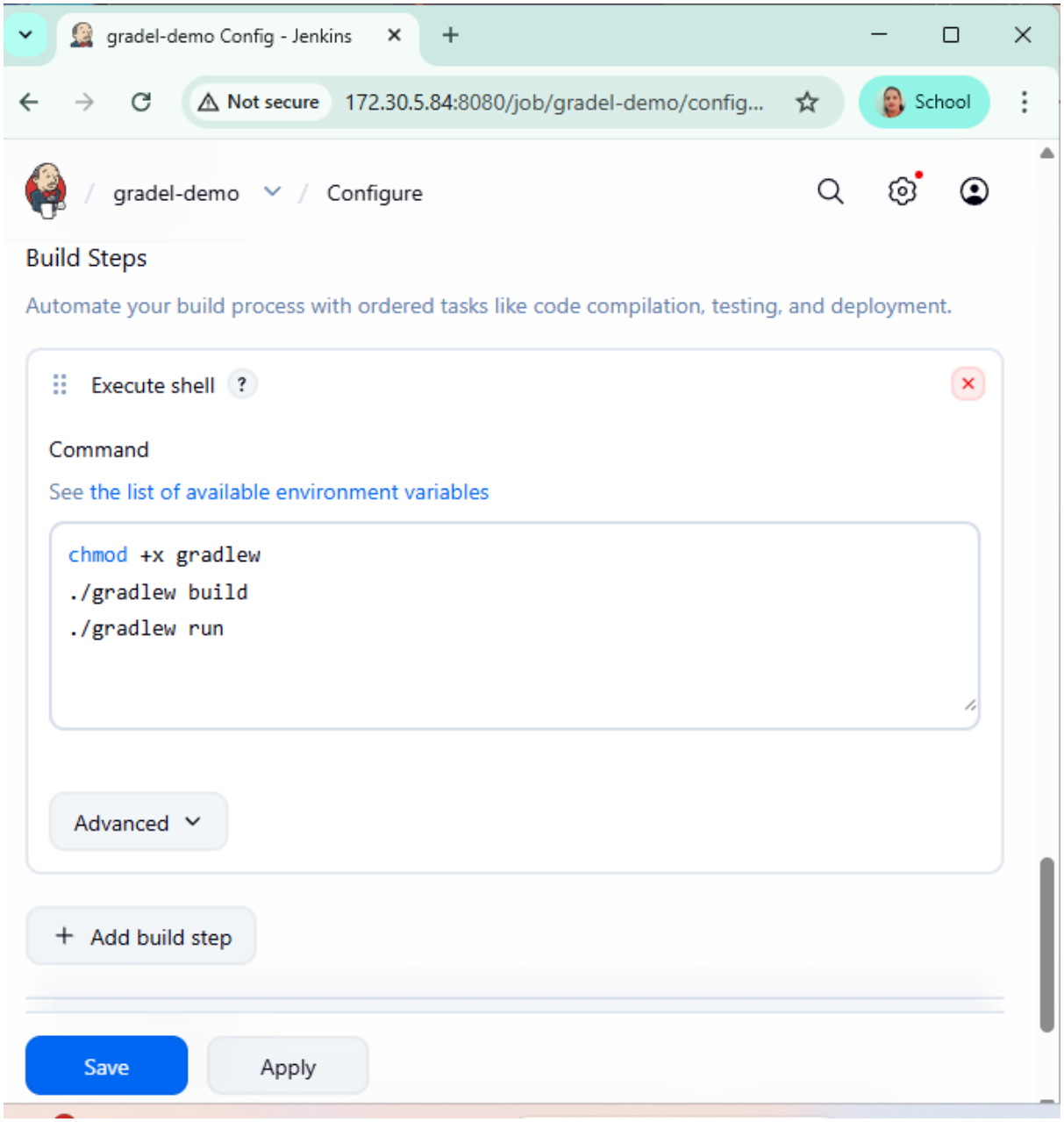
     To find the full path to your `gradle` executable on open terminal in Ubuntu and run:

     ```
     which gradle
     ```



The output will give you the full path (e.g., `/usr/local/bin/gradle` or `/opt/gradle/bin/gradle`). Use this path in the command below.

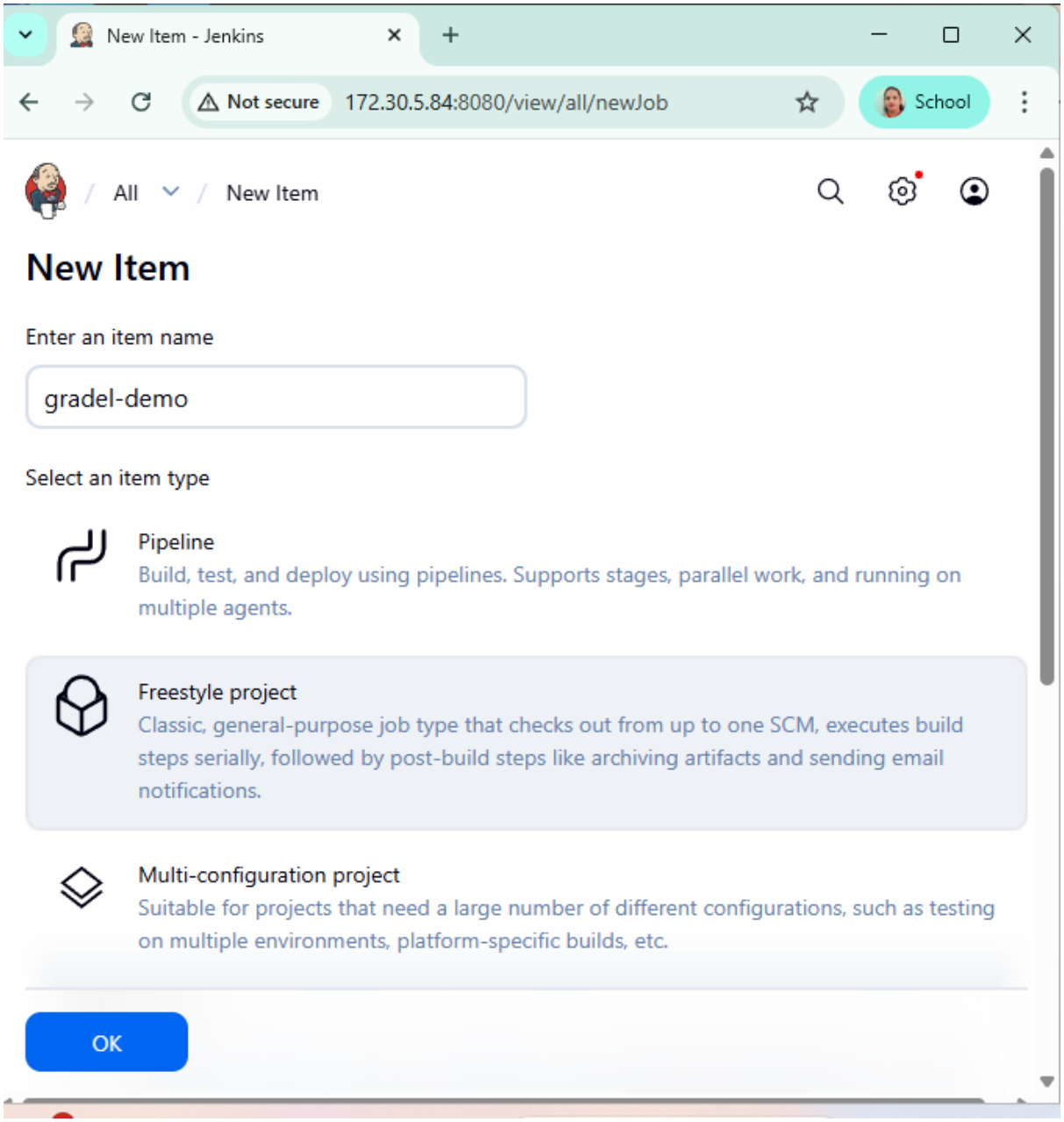
```
#/path/to/your/gradle/bin/gradle build
# Example using the path provided:
/opt/gradle/gradle-8.13/bin/gradle build
```



---

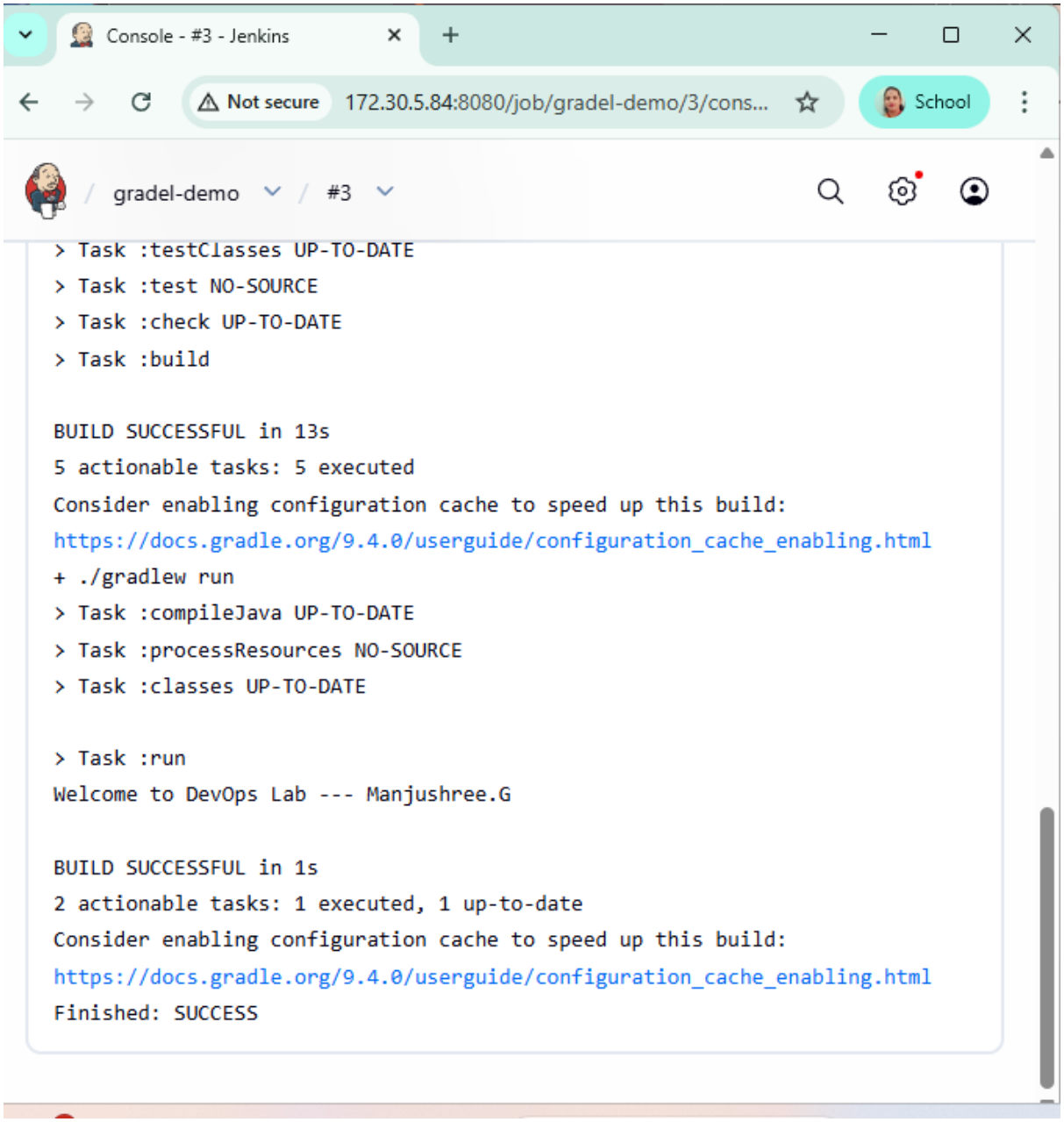
6. Configure Post-Build Actions for Freestyle

To publish the JUnit test reports generated by your build tool so they are visible in the Jenkins UI, follow these steps



:

- Scroll down to the "Post-build Actions" section.
- Click "Add post-build action".
- Select **"Publish JUnit test result report"**.
    - **Test report XMLs:** Enter the path to the test report XML files generated by your build tool.
        - For Maven Surefire/Failsafe: `**/target/surefire-reports/*.xml`
        - For Gradle: `**/test-results/**/*.xml`
    - You can leave other options as default for a basic setup.



---

7. Save the Freestyle Job Configuration

Scroll to the bottom of the page and click "Save".

---

8. Run the Freestyle Jenkins Job

1. You will be redirected to the job's dashboard page.
2. On the left-hand menu, click "Build Now".

Jenkins will now clone your repository, execute the build command, and then process the test report XML files.

---

9. Monitor the Freestyle Build and View Results

1. Under "Build History" on the left, you will see a new build appear with a progress bar.
2. Click on the build number once it starts.
3. Click "Console Output" on the left to view the live build logs.
4. After the build completes, you should see a "Test Result" link on the build's page if test reports were found and published. Click this link to see a summary of your test results.

---

10. Create a New Jenkins Job (Pipeline)

Pipeline is a more modern approach using Groovy scripts to define your build process.

1. On the Jenkins dashboard, click "New Item".
2. Enter an item name (e.g., `Maven-GitHub-Pipeline` or `Gradle-GitHub-Pipeline`).
3. Select "Pipeline".
4. Click "OK".

---

11. Configure the Pipeline Script

In the job configuration page for your Pipeline project:

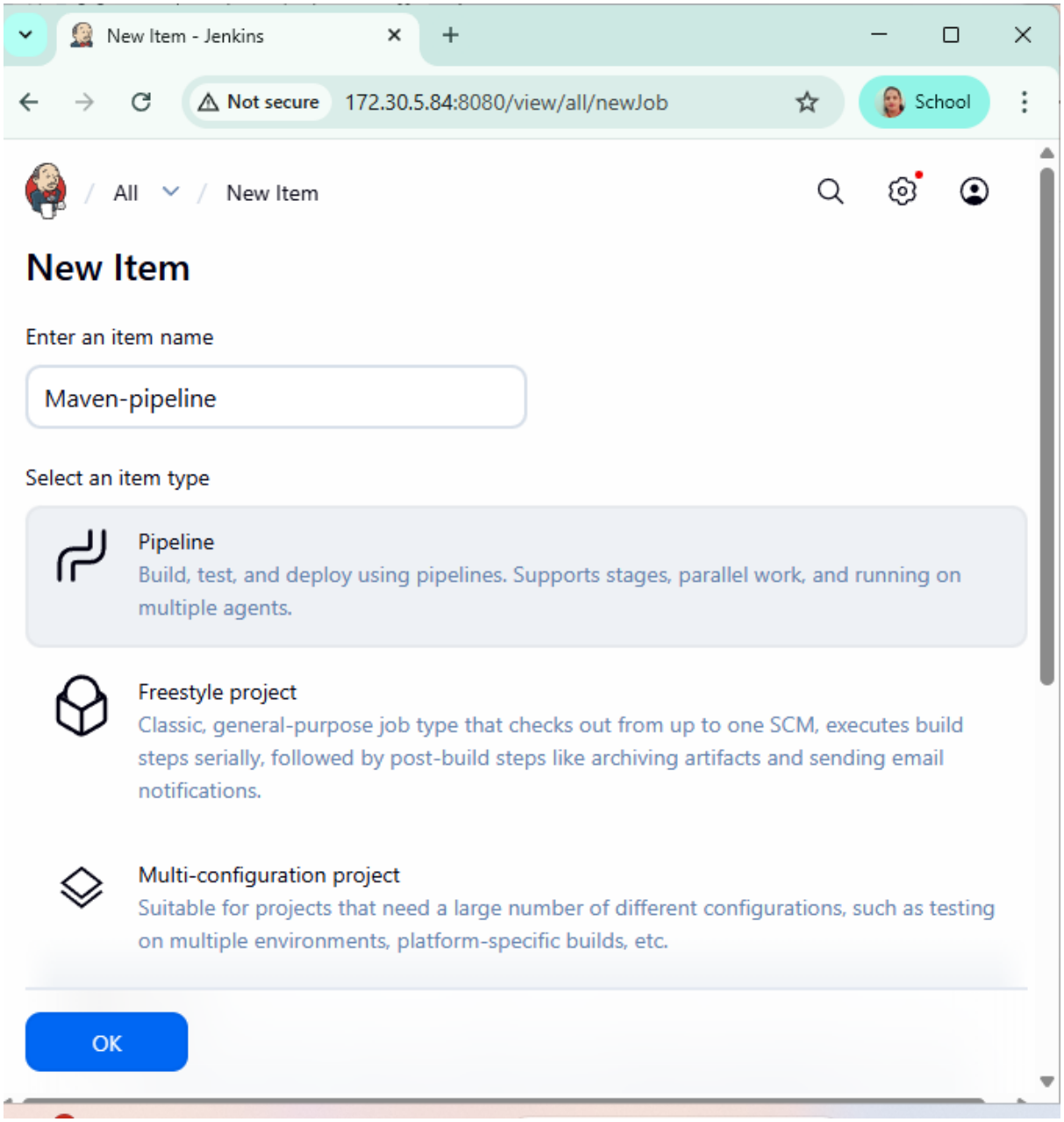
1. Scroll down to the "Pipeline" section.
2. Under "Definition", select **"Pipeline script"**. This allows you to write or paste the Pipeline script directly into the Jenkins job configuration.
3. A text area labeled "Script" will appear. This is where you will paste your Groovy Pipeline script.

---

12. Example Pipeline Scripts to Paste Directly

Here are basic examples of Groovy Pipeline scripts for Maven and Gradle projects that you can paste directly into the "Script" text area in your Jenkins job configuration. These scripts define stages for checking out code, building, testing, and publishing results.

- **Example Pipeline Script for a Maven Project:**

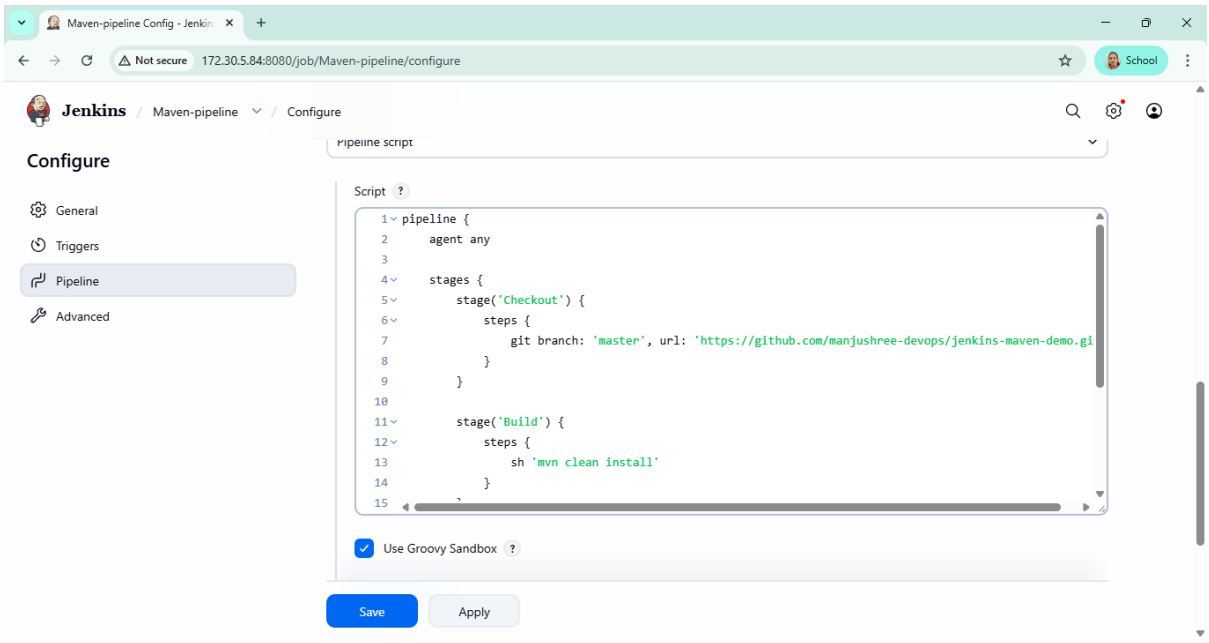


ADD THE BELOW SCRIPT TO THE SECTION

```
pipeline {

    agent any



    stages {

        stage('Checkout') {

            steps {

                git branch: 'master', url: 'https://github.com/manjushree-
devops/jenkins-maven-demo.git'

            }
```

```
        }

        stage('Build') {
            steps {
                sh 'mvn clean install'
            }
        }

        stage('Output') {
            steps {
                sh 'echo "Maven pipeline project by Manjushree G"'
            }
        }
    }
}
```

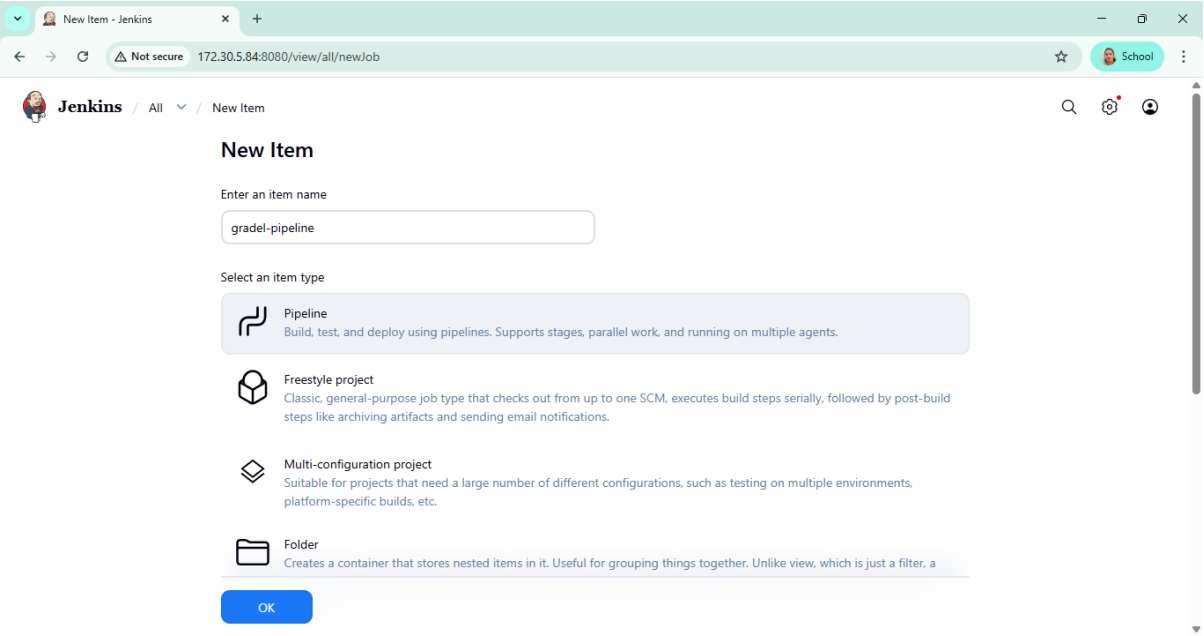
Remember to replace the placeholder paths for `mvn` with the actual full path on your Jenkins agent if you are using the Maven example.



- **Example Pipeline Script for a Gradle Project:**

```
pipeline {

agent any


stages {

stage('Checkout') {

steps {

git branch: 'master', url: 'https://github.com/manjushree-devops/jenkins-gradel-demo.git'

}

}


stage('Build') {

steps {

sh 'chmod +x gradlew'

sh './gradlew build'

}

}


stage('Run') {

steps {

sh './gradlew run'

}

}


stage('Output') {

steps {
```

```
sh 'echo "Gradle pipeline project by Manjushree G'"
}
}
}
}
```



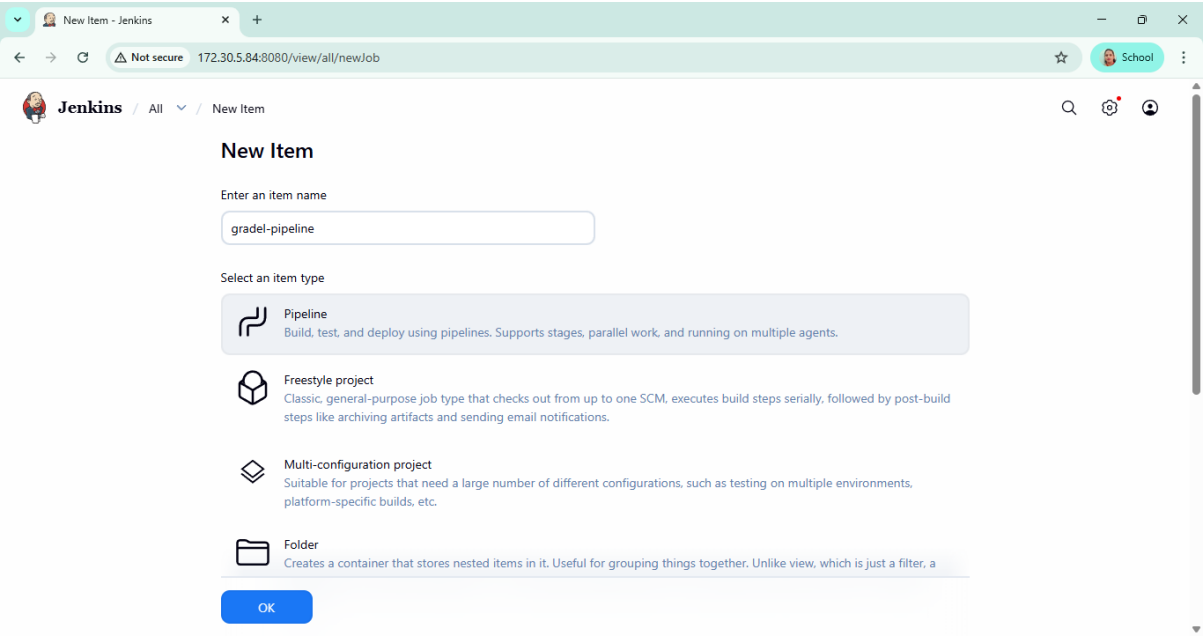
Console - #1 - Jenkins

Not secure 172.30.5.84:8080/job/gradel-pipeline/1/co...

School

gradel-pipeline / #1

```
> Task :classes UP-TO-DATE

> Task :run
Welcome to DevOps Lab --- Manjushree.G

BUILD SUCCESSFUL in 2s
2 actionable tasks: 1 executed, 1 up-to-date
Consider enabling configuration cache to speed up this build:
https://docs.gradle.org/9.4.0/userguide/configuration_cache_enabling.html
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Output)
[Pipeline] sh
+ echo Gradle pipeline project by Manjushree G
Gradle pipeline project by Manjushree G
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



13. Save the Pipeline Job Configuration

Scroll to the bottom of the page and click "Save".

14. Run the Pipeline Jenkins Job

1. You will be redirected to the job's dashboard page.
2. On the left-hand menu, click "Build Now".

Jenkins will now execute the stages defined in the Pipeline script you pasted.

15. Monitor the Pipeline Build and View Results

1. Under "Build History" on the left, you will see a new build appear.
2. Click on the build number.
3. You will see a visual representation of the pipeline stages. Click on a stage to see its details.
4. Click "Console Output" on the left to view the logs for the entire pipeline run.
5. Similar to the Freestyle job, if test reports are published, you will see a "Test Result" link on the build's page.

This covers setting up both Freestyle and basic Pipeline jobs in Jenkins to build Java projects from GitHub and publish test results.

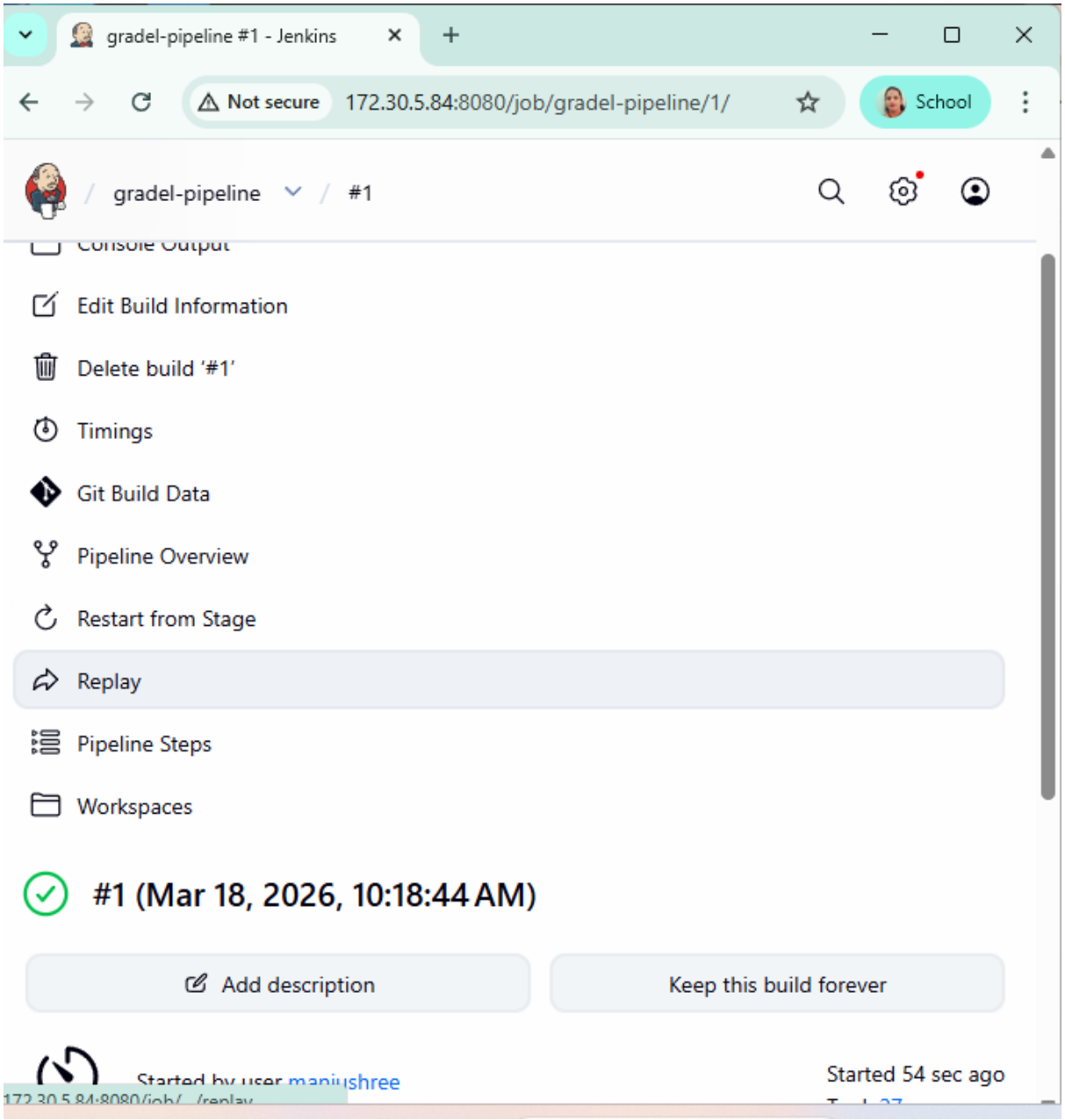
Console - #1 - Jenkins

Not secure 172.30.5.84:8080/job/gradel-pipeline/1/co...

School

gradel-pipeline / #1

```
> Task :classes UP-TO-DATE

> Task :run
Welcome to DevOps Lab --- Manjushree.G

BUILD SUCCESSFUL in 2s
2 actionable tasks: 1 executed, 1 up-to-date
Consider enabling configuration cache to speed up this build:
https://docs.gradle.org/9.4.0/userguide/configuration_cache_enabling.html
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Output)
[Pipeline] sh
+ echo Gradle pipeline project by Manjushree G
Gradle pipeline project by Manjushree G
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```


gradel-pipeline #1 - Jenkins
Not secure
172.30.5.84:8080/job/gradel-pipeline/1/
School
gradel-pipeline
#1
Edit Build Information
Delete build '#1'
Timings
Git Build Data
Pipeline Overview
Restart from Stage
Replay
Pipeline Steps
Workspaces
#1 (Mar 18, 2026, 10:18:44 AM)
Add description
Keep this build forever
Started by user manjushree
Started 54 sec ago

Console - #2 - Jenkins

Not secure 172.30.5.84:8080/job/Maven-pipeline/2/co...

School

Maven-pipeline / #2

```
[1;34mINFO[m] [1m------------------------------------------------------------------------[m
[1;34mINFO[m] [1;32mBUILD SUCCESS[m
[1;34mINFO[m] [1m------------------------------------------------------------------------[m
[1;34mINFO[m] Total time:  2.692 s
[1;34mINFO[m] Finished at: 2026-03-18T10:10:46Z
[1;34mINFO[m] [1m------------------------------------------------------------------------[m
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Output)
[Pipeline] sh
+ echo Maven pipeline project by Manjushree G
Maven pipeline project by Manjushree G
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```